LAB PROGRAMS

Name: S. Aarathana
Reg No: 1921260021

6. VACUUM CLEANER PROBLEM

Aim

To implement the Vacuum Cleaner problem using Python and clean all dirty rooms.

Algorithm

1. Initialize rooms A and B as dirty.
2. Visit each room one by one.
3. Check whether the room is dirty.
4. If dirty, clean the room by changing its state to clean.
5. Display the final state of both rooms.

Program

```
rooms = {"A": 1, "B": 1}

print("Initial State:", rooms)

for room in rooms:
    if rooms[room] == 1:
        print("Room", room, "is Dirty")
        rooms[room] = 0
        print("Room", room, "is Clean")

print("Final State:", rooms)
```

Output

```
Initial State: {'A': 1, 'B': 1}
Room A is Dirty
Room A is Clean
Room B is Dirty
Room B is Clean
Final State: {'A': 0, 'B': 0}
```

Result

Thus, the Vacuum Cleaner problem was successfully implemented and both rooms were cleaned.

7. BREADTH-FIRST SEARCH (BFS)

Aim

To implement the Breadth-First Search (BFS) algorithm using Python.

Algorithm

1. Represent the graph using an adjacency list.
2. Initialize an empty visited list and a queue with the starting node.
3. Remove the first node from the queue.
4. If it has not been visited, mark it as visited.
5. Add its adjacent nodes to the queue.
6. Repeat until the queue is empty and display the traversal.

Program

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

visited = []
queue = ['A']

while queue:
    node = queue.pop(0)

    if node not in visited:
        visited.append(node)
        queue.extend(graph[node])

print("BFS Traversal:", visited)
```

Output

```
========== RESTART: D:/SLOT-2/AI/Exp7.py
BFS Traversal: ['A', 'B', 'C', 'D', 'E', 'F']
```

Result

Thus, the Breadth-First Search algorithm was successfully implemented and the graph was traversed level by level.

8. DEPTH-FIRST SEARCH (DFS)

Aim

To implement the Depth-First Search (DFS) algorithm using Python.

Algorithm

1. Represent the graph using an adjacency list.
2. Initialize an empty visited list.
3. Start the DFS function from node A.
4. Mark the current node as visited.
5. Visit each unvisited adjacent node recursively.
6. Display the nodes in traversal order.

Program

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

visited = []

def dfs(node):
    if node not in visited:
        visited.append(node)

        for next_node in graph[node]:
            dfs(next_node)

dfs('A')

print("DFS Traversal:", visited)
```

Output

```
========================= RESTART: D:/SLOT-2/AI/Ex
DFS Traversal: ['A', 'B', 'D', 'E', 'C', 'F']
>
```

Result

Thus, the Depth-First Search algorithm was successfully implemented using recursion.

9. TRAVELLING SALESMAN PROBLEM

Aim

To find the shortest route that visits every city once and returns to the starting city.

Algorithm

1. Import the permutations function.
2. Define the cities and the distances between them.
3. Generate every possible order of cities except the starting city.
4. Add city A at the beginning and end of each route.
5. Calculate the total cost of each route.
6. Display the route with the minimum cost.

Program

```
from itertools import permutations

cities = ['A', 'B', 'C']

distance = {
    ('A', 'B'): 10, ('A', 'C'): 15,
    ('B', 'A'): 10, ('B', 'C'): 20,
    ('C', 'A'): 15, ('C', 'B'): 20
}

shortest = None
min_cost = float('inf')

for route in permutations(cities[1:]):
    path = ['A'] + list(route) + ['A']
    cost = 0

    for i in range(len(path) - 1):
        cost += distance[(path[i], path[i + 1])]

    if cost < min_cost:
        min_cost = cost
        shortest = path

print("Shortest Route:", shortest)
print("Minimum Cost:", min_cost)
```

Output

```
>>
========================= RESTART: D:/SLOT-2/AI
Shortest Route: ['A', 'B', 'C', 'A']
Minimum Cost: 45
>>
```

Result

Thus, the Travelling Salesman Problem was implemented and a minimum-cost route was obtained.

10. A SEARCH ALGORITHM*

Aim

To implement the A* search algorithm in Python to find a path from the start node to the goal node.

Algorithm

1. Represent the graph with edge costs.
2. Define the heuristic value for each node.
3. Initialize the open list with the starting node.
4. Select the node with the lowest estimated total cost.
5. If the goal is reached, display the path and its cost.
6. Otherwise, add adjacent nodes with updated costs and repeat.

Program

```
graph = {
    'A': [('B', 1), ('C', 3)],
    'B': [('D', 1)],
    'C': [('D', 1)],
    'D': []
}

h = {'A': 2, 'B': 1, 'C': 1, 'D': 0}

open_list = [(h['A'], 0, 'A', ['A'])]
visited = []

while open_list:
    open_list.sort()
    f, g, node, path = open_list.pop(0)

    if node == 'D':
        print("Shortest Path:", path)
        print("Total Cost:", g)
        break

    if node not in visited:
        visited.append(node)

        for next_node, cost in graph[node]:
            new_g = g + cost
            new_f = new_g + h[next_node]
            open_list.append(
                (new_f, new_g, next_node, path + [next_node])
            )
```

Output

```
>>>
========================= RESTART: D:/SLOT-2/AI/
Shortest Path: ['A', 'B', 'D']
Total Cost: 2
>>>
```

Result

Thus, the A* search algorithm was successfully implemented to find a path from the start node to the goal node.